

FieldProof

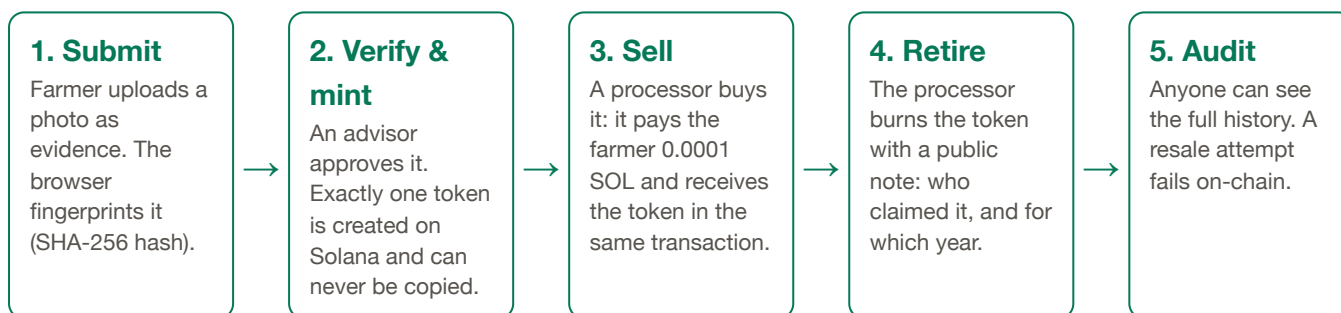
How the app works: a plain-language guide to every screen, every transaction and every number

In one sentence: FieldProof turns one verified farm climate action into a single digital token on Solana. The token can be sold once and then permanently destroyed ("retired"), so the same action can never be paid for or counted twice.

The problem it solves

When a farmer does something that cuts emissions (for example, switching fertiliser), several parties may want to claim that reduction: the farmer, a co-op, a milk processor, a food brand. Without a shared record, the same tonne can be counted in two reports, or paid for twice. FieldProof gives each action a public, tamper-proof life story: **verified** → **minted** → **sold** → **retired**. Once retired, anyone can check that it is gone for good.

1. The journey of one climate action



The website shows this journey through three role tabs (**Farmer**, **Verifier**, **Processor**) and an **Audit trail** that is always visible underneath. There are no logins: in the demo, you play every role yourself by switching tabs.

Statuses an action can have

Status	Meaning
SUBMITTED	Farmer has sent it in, waiting for the verifier.
REJECTED	Verifier said the evidence wasn't good enough. The farmer can resubmit with a new photo, which puts it back to <i>submitted</i> .
APPROVED	Verifier approved it and the record hash is fixed. Normally this lasts about a second, until minting finishes. If minting failed (e.g. no SOL), it stays here and "Approve & mint" retries.
MINTED	One token exists on Solana, held by the farmer. Ready to sell.
SOLD	Token has moved to the processor and the farmer was paid. Normally only lasts until the burn completes.
RETIRED	Token is burned. The reduction is claimed for good by the named buyer and year.

2. The screens, one by one

Header

Shows the slogan and a badge: **Mock mode** (yellow, nothing touches the blockchain) or **Solana devnet** (green, real transactions on Solana's free test network). See section 6.

Farmer tab: submit an action

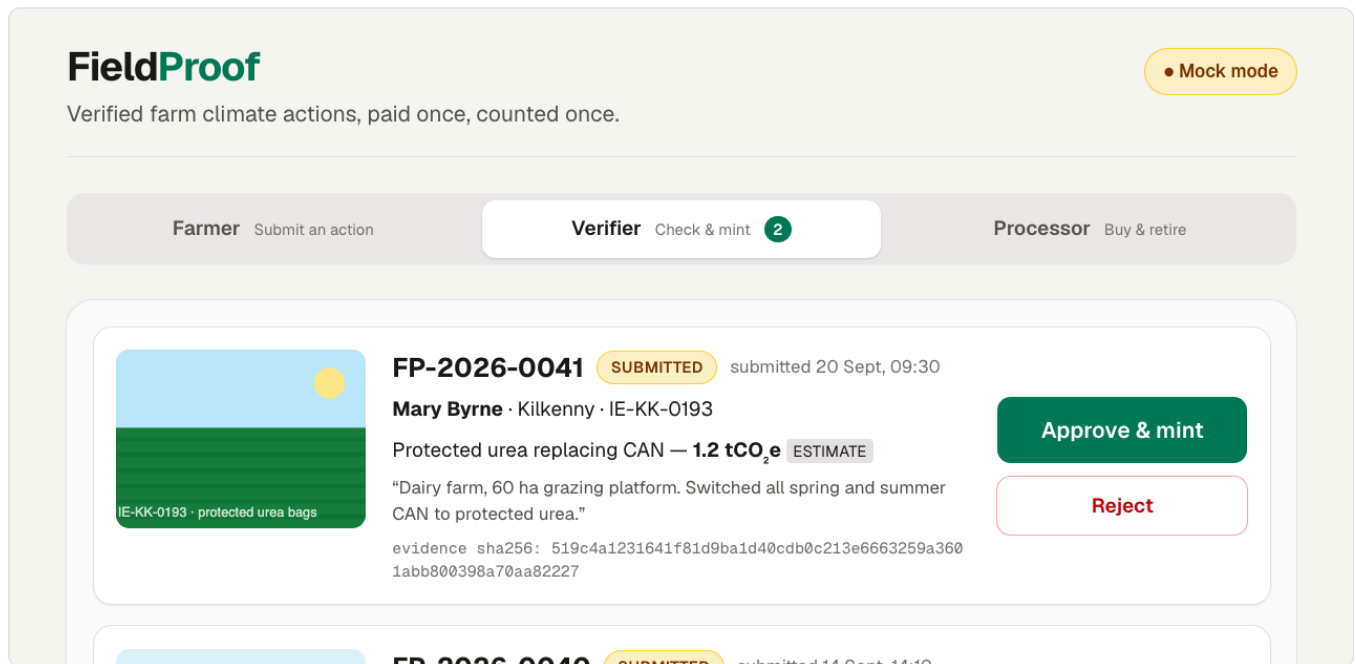
The screenshot shows the 'FieldProof' interface in 'Mock mode'. At the top, a banner reads 'Verified farm climate actions, paid once, counted once.' Below this is a navigation bar with three tabs: 'Farmer' (active, with 'Submit an action'), 'Verifier' (with 'Check & mint' and a green circle with '2'), and 'Processor' (with 'Buy & retire'). The main form area is divided into three sections. The 'Farm' section has a dropdown menu showing 'Mary Byrne · Kilkenny · IE-KK-019'. The 'Practice' section has a dropdown menu showing 'Protected urea replacing CAN' and a text display showing '1.2 tCO₂e' with a grey 'ESTIMATE' tag and 'season 2026'. The 'Note' section has a text area with the placeholder text 'What was done, where and when (e.g. 'Switched all CAN to protected urea on 60 ha')'. On the right, an 'Evidence preview' section shows a large grey box with the text 'No photo yet'.

Farmer tab. The evidence hash appears on the right as soon as a photo is chosen.

- **Farm:** one of four demo Irish farms (Mary Byrne, Kilkenny; Seán Murphy, Tipperary; Aoife Kelly, Cork; Declan Walsh, Wexford).
- **Practice:** three options, each with a fixed **estimate** of the reduction: Protected urea replacing CAN (1.2 tCO₂e), Low-emission slurry spreading (0.8), Clover-rich sward reseeding (1.5). These are always labelled "estimate". They are not measured values.

- **Note:** free text describing what was done.
- **Evidence photo:** when you pick an image, your browser computes its **SHA-256 hash**, a 64-character fingerprint. Change even one pixel and the hash changes completely. A small thumbnail is stored for the verifier to look at. The hash is computed from the original file.
- **Submit:** creates a new record with an ID like `FP-2026-0042` and status *submitted*.
- **Resubmit:** if the chosen farm has a rejected action, a red box offers "Resubmit with new evidence". This reuses the same action ID with a new photo and hash.

Verifier tab: check the evidence and mint



Verifier tab: each waiting action shows the photo, farm, practice, estimate and evidence hash.

- Lists every action with status *submitted*. The verifier is a mock "co-op advisor".
- **Reject:** marks it *rejected* with a reason, and nothing happens on-chain.
- **Approve & mint:** the server computes the **record hash** (see section 4), then creates the token on Solana. While it runs, the card shows "**Minting on Solana...**". When done, a green **✓ Minted** banner appears with links to the transaction and the token on Solana Explorer.

Processor tab: buy and retire

FieldProof

• Mock mode

Verified farm climate actions, paid once, counted once.

Farmer Submit an action

Verifier Check & mint 1

Processor Buy & retire 1

FP-2026-0041 Mary Byrne · Kilkenney
Protected urea replacing CAN — 1.2 tCO₂e **ESTIMATE**

mock mint mock tx

Buyer

Glanbia Mock Processor

Reporting year

FY2026

☐ Transferring... (pays farmer 0.0001 SOL)

☐ Burning with memo...

Buy and retire

Processor tab: buyer name and reporting year go into the permanent retirement note.

- Lists every *minted* action. You can edit the **Buyer** (default "Glanbia Mock Processor") and **Reporting year** (default FY2026).
- **Buy and retire** runs two steps, shown live:
 1. **"Transferring..."**: the processor pays the farmer 0.0001 SOL, and the token moves from the farmer to the processor, all in one transaction.
 2. **"Burning with memo..."**: the processor destroys the token and writes the note **RETIRED** | **FP-2026-0041** | **Glanbia Mock Processor** | **FY2026** on-chain.
- If the burn fails after the transfer succeeded, the button changes to **"Finish retiring"** and only redoes the burn.

Audit trail: visible on every tab

Audit trail

Every action, every state change, every on-chain transaction.

FP-2026-0041 **RETIRED** Mary Byrne · Kilkenney · Protected urea replacing CAN · 1.2 tCO₂e (estimate)

mock mint

Try to resell

Submitted
20 Sept, 09:30

Approved
26 Sept, 15:04

Minted
26 Sept, 15:04
mock tx

Sold
26 Sept, 15:04
mock tx

Retired
26 Sept, 15:04
mock tx

evidence hash 519c4a1231641f81d9ba1d40cdb0c213e6663259a3601abb800398a70aa82227

record hash 35215895e3a29cd340e9bf32c84447fff9981dc8709a03eeea361364d74145bd

farmer paid 0.0001 SOL by Glanbia Mock Processor (in the sale tx)

retired for Glanbia Mock Processor · FY2026

A retired action: every step has a timestamp and a transaction link. "Try to resell" proves the token is gone.

- One card per action with a timeline: submitted → approved → minted → sold → retired (or submitted → rejected).

- Each on-chain step shows a **link to Solana Explorer** (`explorer.solana.com/tx/...?cluster=devnet`), plus a link to the token ("mint") itself.
- Link labels: a green link is a real devnet transaction, "**mock tx**" is a fake one from mock mode, and "**seed data**" is pre-loaded demo history that was never on-chain.
- Also shows the evidence hash, record hash, the SOL paid to the farmer, and who retired it for which year.
- **Try to resell** (retired actions only): the app really tries to sell the token again. Solana rejects it, and a red box shows "**Resale failed: record already retired (supply 0, mint authority none)**" together with the actual error from the Solana Token program.

3. What happens on the blockchain

FieldProof writes no custom smart contract. It only uses two standard programs that already exist on every Solana network: the **SPL Token program** (creates, moves and burns tokens) and the **Memo program** (attaches a permanent text note to a transaction). All transactions are built and signed **on the server**, inside the Next.js API routes, so private keys never reach the browser and nobody sees a wallet pop-up.

The three accounts

Account	Role	Holds SOL?
Issuer	Runs the scheme. Creates each token, is the temporary "mint authority", and pays every fee and deposit.	Yes: the only account you fund (e.g. 0.1 SOL from Phantom).
Farmer	Receives the token when minted, signs to hand it over when selling, receives the 0.0001 SOL payment.	Topped up once by the issuer to the minimum balance a Solana wallet needs.
Processor	Buys the token, pays the farmer, burns the token.	Topped up once by the issuer with enough for about 10 payments.

The secret keys of all three live in `.env.local`. They are throwaway demo keys for devnet. Your Phantom wallet is only used to send SOL to the issuer. The app never asks Phantom to sign anything.

Step by step

App step	Solana transaction(s)	Signed by
Approve & mint	(a) Create a new token "mint" with 0 decimals (only whole tokens) and no freeze authority . (b) Create the farmer's token account for it. (c) One transaction that: mints exactly 1 token to the farmer, then removes the mint authority (sets it to none, so no second token can ever be made), then adds the memo <code>FP-2026-0041 <record hash></code> .	Issuer
Buy (transfer)	First sale only: a small top-up transaction from the issuer to the farmer and processor wallets. Then one transaction that: sends 0.0001 SOL processor → farmer, creates the processor's token account, moves the token farmer → processor, and closes the farmer's now-empty token account (its deposit goes back to the issuer).	Issuer (fee payer), farmer, processor
Retire (burn)	One transaction that: burns the 1 token, closes the processor's now-empty token account (deposit back to the issuer), and adds the memo <code>RETIRED id buyer year</code> .	Issuer (fee payer), processor
Try to resell	Builds a transaction that tries to move the token again. Solana's check (simulation) fails with <i>"insufficient funds"</i> (custom program error 0x1), so it is never sent and costs nothing. The app then reads the mint and shows supply 0 and mint authority none .	n/a (rejected before sending)

Why this proves "paid once, counted once"

- **Only one can ever exist.** Exactly 1 token is minted, and in the same transaction the power to mint more is thrown away. Anyone can check on Explorer that the mint authority is "none".
- **Only one owner at a time.** A token can't be in two wallets, so it can't be sold to two buyers.
- **Retired means gone.** After the burn, the supply is 0. The resale attempt fails at the blockchain level, not because the website refuses it.
- **The claim is public.** The burn memo records who claimed the reduction and for which reporting year, permanently.
- **Payment and handover are atomic.** The farmer's payment and the token transfer are in the same transaction: both happen, or neither does.

4. The two fingerprints (hashes)

Evidence hash

SHA-256 of the **original photo file**, computed in the farmer's browser. It proves which exact photo was submitted. If someone later swaps the photo, its hash no longer matches.

Record hash

SHA-256 of a fixed-format JSON text containing `actionId`, `farmId`, `practice`, `season`, `verifier`, `evidenceHash`, computed on the server at approval. It is written on-chain in the mint memo, so the approved details can't be quietly changed afterwards.

To check a record: rebuild that JSON from the stored data, hash it, and compare the result with the memo on Explorer. A match means nothing was altered since approval.

5. What it costs (devnet SOL)

Item	Amount	Comes back?
Token mint account deposit ("rent")	≈ 0.00146 SOL	No. It stays with the mint, which acts as the permanent public record.
Farmer token account deposit	≈ 0.00204 SOL	Yes, returned to the issuer when it is closed after the sale.
Processor token account deposit	≈ 0.00204 SOL	Yes, returned to the issuer when it is closed after the burn.
Payment to farmer	0.0001 SOL	No. It is the farmer's payment (setting: <code>PAYMENT_SOL</code>).
Transaction fees	≈ 0.000005 SOL per signature	No
One-time wallet top-ups (first sale only)	≈ 0.0028 SOL	Stays in the farmer/processor wallets for later runs.

A full run costs about 0.0016 SOL net. 0.1 SOL is enough for roughly 50 demo runs. A Solana wallet needs about 0.00089 SOL before it can receive a small payment, which is why the one-time top-up exists.

6. Mock mode vs Solana devnet

	Mock mode (<code>MOCK_MODE=true</code>)	Devnet (<code>MOCK_MODE=false</code>)
Blockchain	Not touched. Each step waits about 1 second.	Real transactions on Solana devnet (the free test network, no real money).
Signatures & links	Fake IDs starting with <code>MOCK_</code> , shown as "mock tx".	Real signatures with clickable Explorer links.
Needs keys / SOL	No	Yes: three keys in <code>.env.local</code> and a funded issuer.
When to use	Rehearsing, or as a fallback if Wi-Fi or devnet fails on stage.	The real demo.

The screens behave the same in both modes. To switch, edit `MOCK_MODE` in `.env.local` and restart `npm run dev`. The app refuses to run against mainnet.

7. Where the data lives

All actions are stored in one local file, `data/actions.json`. Each record holds the farm and farmer, practice, note, season, estimate, evidence hash and thumbnail, verifier, status, record hash, the token (mint) address, the three transaction signatures (mint, transfer, burn), the buyer, reporting year, SOL paid, and a **history** list of every status change with its timestamp and transaction.

Four demo actions are pre-loaded so the page is never empty: Mary Byrne (*submitted*, ready for the live demo), Declan Walsh (*submitted*), Aoife Kelly (*rejected*, blurry photo) and Seán Murphy (*retired*, seed history only). Run `npm run reset` to restore them between rehearsals. This resets only the local file; anything already on devnet stays there.

Behind the screens: the server endpoints

Endpoint	What it does
<code>GET /api/actions</code>	Returns all records (the page loads this).
<code>POST /api/actions</code>	Creates a new submitted action, or resubmits a rejected one.
<code>POST /api/verify</code>	Approve (record hash + mint) or reject.
<code>POST /api/retire</code>	Sale transfer with payment, then burn with memo.
<code>POST /api/resell</code>	Attempts the resale and returns the real error plus the mint's supply and authority.

8. The 90-second demo

1. **Farmer:** pick Mary Byrne, choose "Protected urea replacing CAN", upload a photo and point out the hash, then Submit. *(Or skip ahead: FP-2026-0041 is already waiting.)*
2. **Verifier:** show the photo, hash and estimate, then click **Approve & mint**. Open the Explorer link and point out the memo and that mint authority is "none".
3. **Processor:** click **Buy and retire** and watch "Transferring..." then "Burning with memo...". Mention that the farmer got paid in the same transaction.
4. **Audit trail:** walk the timeline, then click **Try to resell** and show the red "Resale failed" message with the real Solana error.

What is simplified in this MVP

- The verifier is a mock co-op advisor, and there are no logins or real identities.
- Reduction figures are fixed practice-based estimates, not measurements.
- All three keys sit on one server (custodial), which is fine for a demo but not for production.
- Data is a local JSON file, not a shared database. Only hashes and memos are on-chain, not the photos.
- Devnet only: tokens and SOL have no real-world value.